MedTech
Mediterranean
Institute of Technology

CS202 – ADVANCED OO PROGRAMMING

CHAPTER 7
CLIENT-SERVER COMMUNICATION IN JAVA

SOPHOMORE SE
SPRING 2026

CS Communication

- Java Networking
- Socket programming
- Server and Socket Java classes
- Examples

Java Networking

The term **network programming** refers to writing programs that execute across multiple devices (computers), in which the devices are all connected to each other using a network.

The **java.net** package of the J2SE APIs contains a collection of classes and interfaces that provide the low-level communication details, allowing the programmer to write programs that focus on solving the problem at hand.

This package provides support for the two common network protocols –

- **TCP – TCP** Transmission Control Protocol, which allows for reliable communication between two applications. TCP is typically used over the Internet Protocol, which is referred to as TCP/IP.
- **UDP – UDP** User Datagram Protocol, a connection-less protocol that allows for packets of data to be transmitted between applications.

Two interesting approaches:

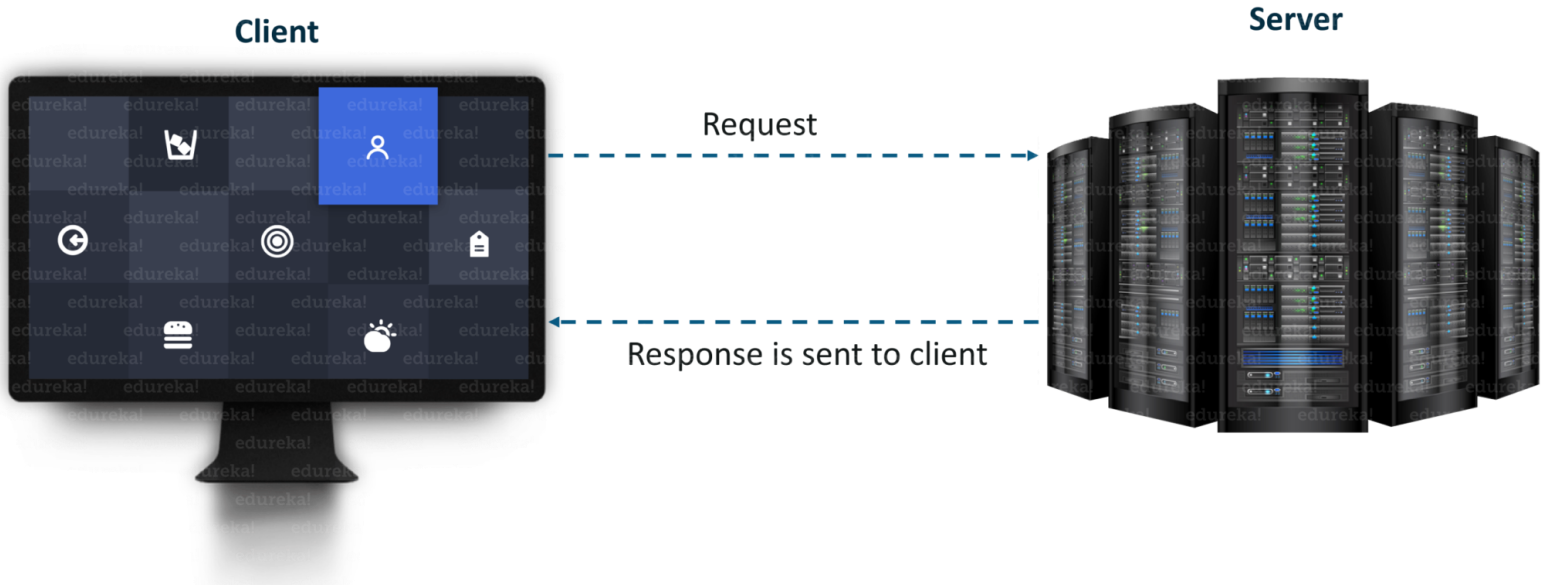
- **Socket Programming –** (most widely used concept in Java Networking)
- **URL Processing**

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

What is Socket Programming in Java?

Socket programming in Java is used for communication between the applications that are running on different JRE.

Socket programming is a way of connecting two nodes on a network to communicate with each other.



What is a socket?

A **socket** in Java is **one endpoint** of two-way communication link between two programs running on the network.

An endpoint is a combination of an **IP address** and a **port number**.

The **java.net** package provides a class **Socket** that implements one side of a two-way connection between the Java program and another program on the network.

On the whole, a socket is a way to establish a connection between a client and a server.

One socket (node) listens on a particular port at an IP, while the other socket reaches out to the other in order to form a connection.

A socket is linked to a port number allowing the TCP layer the identification of which application that data is supposed to be sent.

The listener socket represents the server while the client reaches out to the server. **Socket** and **ServerSocket** classes are used for connection-oriented socket programming.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Socket programming: How does it operate? (1/3)

Sockets provide the communication mechanism between two computers using TCP.

A client program creates a socket on its end of the communication and attempts to connect that socket to a server.

When the connection is made, the server creates a socket object on its end of the communication. The client and the server can now communicate by writing to the socket and reading from it.

The **java.net.Socket** class represents a socket, and the **java.net.ServerSocket** class provides a mechanism for the server program to listen for clients and establish connections with them.

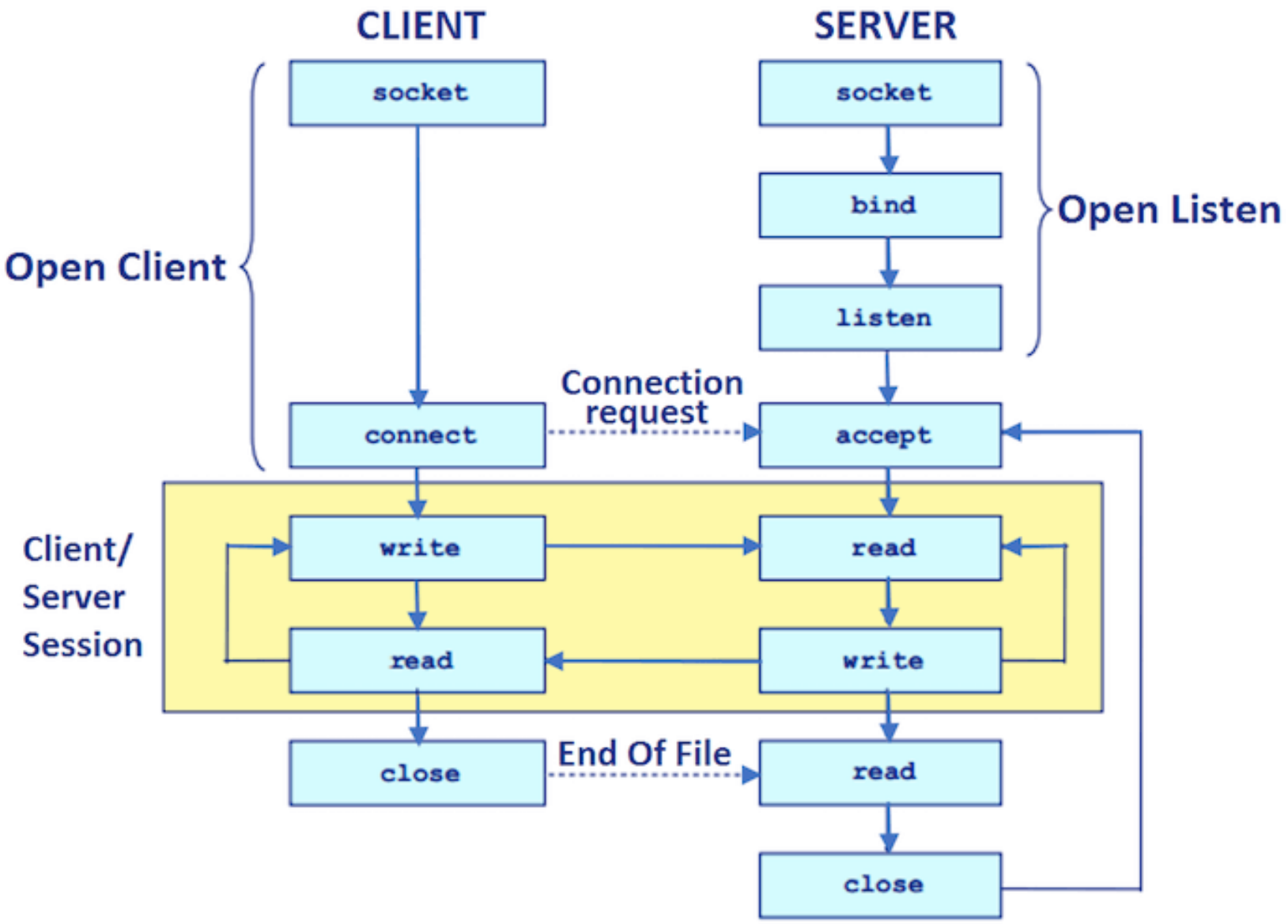
SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Socket programming: How does it operate? (2/3)

Steps for establishing a **TCP connection** between **two computers** using **sockets**

- The **server** instantiates a **ServerSocket** object, denoting which communication **port number** has to be used.
- The **server** invokes the **accept()** method of the **ServerSocket** class. This method waits until a client connects to the server on the given port.
- After the server is waiting, a client instantiates a **Socket** object, specifying the **server name** and the **port number** to connect to.
- The constructor of the Socket class attempts to connect the **client** to the specified **server** and the port number. If communication is established, the client now has a Socket object capable of communicating with the server.
- On the server side, the **accept()** method returns a reference to a new socket on the server that is connected to the client's socket.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues


CLIENT
SERVER
socket
socket
bind
listen
Open Listen
Open Client
Connection request
connect
accept
Client/ Server Session
write
read
read
write
End Of File
close
read
close

SOCKET API

Socket programming: How does it operate? (3/3)

- After the connections are established, communication can occur using I/O streams.
- Each socket has both an **OutputStream** and an **InputStream**.
- The client's OutputStream is connected to the server's InputStream, and the client's InputStream is connected to the server's OutputStream.
- TCP is a **two-way** communication protocol, hence data can be sent across both streams at the same time.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

ServerSocket Class

The **java.net.ServerSocket** class is used by server applications to obtain a port and listen for client requests.

A server socket waits for requests to come in over the network. It performs some operations based on that request, and then possibly returns a result to the requester.

It has four constructors:

- **public ServerSocket(int port) throws IOException :** Creates a server socket bound to the specified port. An exception occurs if the port is already bound by another application.
- **public ServerSocket(int port, int backlog) throws IOException:** Similar to the previous constructor, the backlog parameter specifies how many incoming clients to store in a wait queue.
- **public ServerSocket(int port, int backlog, InetAddress address) throws IOException :** the InetAddress parameter specifies the local IP address to bind to. The InetAddress is used for servers that may have multiple IP addresses, allowing the server to specify which of its IP addresses to accept client requests on.
- **public ServerSocket() throws IOException:** Creates an unbound server socket. When using this constructor, use the bind() method when you are ready to bind the server socket.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

ServerSocket Class Methods

If the ServerSocket constructor does not throw an exception, it means that the application has successfully bound to the specified port and is ready for client requests.

public int getLocalPort() Returns the port that the server socket is listening on. This method is useful if we passed in 0 as the port number in a constructor and let the server find a port for us.

public Socket accept() throws IOException Waits for an incoming client. This method blocks until either a client connects to the server on the specified port or the socket times out, assuming that the time-out value has been set using the setSoTimeout() method. Otherwise, this method blocks indefinitely.

public void setSoTimeout(int timeout) Sets the time-out value for how long the server socket waits for a client during the accept().

public void bind(SocketAddress host, int backlog) Binds the socket to the specified server and port in the SocketAddress object. This method can be used, if we have instantiated the ServerSocket using the no-argument constructor.

When the ServerSocket invokes accept(), the method does not return until a client connects. After a client does connect, the ServerSocket creates a new Socket on an unspecified port and returns a reference to this new Socket. A TCP connection will be established between the client and the server, and communication can begin.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Socket Class

The **java.net.Socket** class represents the socket that both the client and the server use to communicate with each other. The client obtains a Socket object by instantiating one, whereas the server obtains a Socket object from the return value of the accept() method.

The Socket class has five constructors that a client uses to connect to a server

- **Socket(String host, int port) throws UnknownHostException, IOException.** This method attempts to connect to the specified server at the specified port. If this constructor does not throw an exception, the connection is successful, and the client is connected to the server.
- **Socket(InetAddress host, int port) throws IOException** This method is identical to the previous constructor, except that the host is denoted by an InetAddress object.
- **Socket(String host, int port, InetAddress localAddress, int localPort) throws IOException.** Connects to the specified host and port, creating a socket on the local host at the specified address and port.
- **Socket(InetAddress host, int port, InetAddress localAddress, int localPort) throws IOException.** This method is identical to the previous constructor, except that the host is denoted by an InetAddress object instead of a String.
- **Socket()** Creates an unconnected socket. Use the connect() method to connect this socket to a server.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Socket Class Methods

When the Socket constructor returns, it does not simply instantiate a Socket object but it actually attempts to connect to the specified server and port.

Some methods of interest in the Socket class are listed here. Notice that both the client and the server have a Socket object, so these methods can be invoked by both the client and the server.

public void connect(SocketAddress host, int timeout) throws IOException This method connects the socket to the specified host. This method is needed only when you instantiate the Socket using the no-argument constructor.

public InetAddress getInetAddress() This method returns the address of the other computer that this socket is connected to.

public int getPort() Returns the port the socket is bound to on the remote machine.

public int getLocalPort() Returns the port the socket is bound to on the local machine.

public SocketAddress getRemoteSocketAddress() Returns the address of the remote socket.

public InputStream getInputStream() throws IOException Returns the input stream of the socket. The input stream is connected to the output stream of the remote socket.

public OutputStream getOutputStream() throws IOException Returns the output stream of the socket. The output stream is connected to the input stream of the remote socket.

public void close() throws IOException Closes the socket, which makes this Socket object no longer capable of connecting again to any server.

InetAddress Class Methods

This class represents an Internet Protocol (IP) address. Here are following usefull methods which can be used for socket programming

static InetAddress getByAddress(byte[ ] addr) Returns an InetAddress object given the raw IP address.

static InetAddress getByAddress(String host, byte[ ] addr) Creates an InetAddress based on the provided host name and IP address.

static InetAddress getByName(String host) Determines the IP address of a host, given the host's name.

String getHostAddress() Returns the IP address string in textual presentation.

String getHostName() Gets the host name for this IP address.

Static InetAddress getLocalHost() Returns the local host.

String toString() Converts this IP address to a String.

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Socket Programming example 1

```java
import java.io.*;
import java.net.*;
public class MyServer {
    public static void main(String[] args){
        try{
            ServerSocket ss=new ServerSocket(6666);
            Socket s=ss.accept();//establishes connection
            DataInputStream dis=new DataInputStream(s.getInputStream());
            String  str=(String)dis.readUTF();
            System.out.println("message= "+str);
            ss.close();
        }catch(Exception e){System.out.println(e);}
    }
}
```

```java
import java.io.*;
import java.net.*;
public class MyClient {
    public static void main(String[] args) {
        try{
            Socket s = new Socket("localhost",6666);
            DataOutputStream dout=new DataOutputStream(s.getOutputStream());
            dout.writeUTF("Hello Server");
            dout.flush();
            dout.close();
            s.close();
        }catch(Exception e)
            {System.out.println(e);}
}  }
```

SMU
SOUTH MEDITERRANEAN UNIVERSITY
MSB | MedTech | LCI
Business Ingéniorat Langues

Socket client example 2

```
import java.net.*;
import java.io.*;

public class GreetingClient {
   public static void main(String [] args) {
      String serverName = args[0];
      int port = Integer.parseInt(args[1]);
      try {
         System.out.println("Connecting to " + serverName + " on port " + port);
         Socket client = new Socket(serverName, port);

         System.out.println("Just connected to " + client.getRemoteSocketAddress());
         OutputStream outToServer = client.getOutputStream();
         DataOutputStream out = new DataOutputStream(outToServer);

         out.writeUTF("Hello from " + client.getLocalSocketAddress());
         InputStream inFromServer = client.getInputStream();
         DataInputStream in = new DataInputStream(inFromServer);

         System.out.println("Server says " + in.readUTF());
         client.close();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

Socket Server example 2

```
import java.net.*;
import java.io.*;

public class GreetingServer extends Thread {
   private ServerSocket serverSocket;
   public GreetingServer(int port) throws IOException {
      serverSocket = new ServerSocket(port);
      serverSocket.setSoTimeout(10000);}
   public void run() {
      while(true) {
         try {
            System.out.println("Waiting for client on port " + serverSocket.getLocalPort() + "...");
            Socket server = serverSocket.accept();
            System.out.println("Just connected to " + server.getRemoteSocketAddress());
            DataInputStream in = new DataInputStream(server.getInputStream());
            System.out.println(in.readUTF());
            DataOutputStream out = new DataOutputStream(server.getOutputStream());
            out.writeUTF("Thank you for connecting to " + server.getLocalSocketAddress() + "\nGoodbye!");
            server.close();}
         catch (SocketTimeoutException s) {
            System.out.println("Socket timed out!");
            break; }
         catch (IOException e) {
            e.printStackTrace();
            break;}}
}
```

```
public static void main(String [] args) {
      int port = Integer.parseInt(args[0]);
      try {
         Thread t = new GreetingServer(port);
         t.start();
      } catch (IOException e) {
         e.printStackTrace();
      }
   }
}
```

Example 3

```
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.lang.ClassNotFoundException;
import java.net.ServerSocket;
import java.net.Socket;

public class SocketServerExample {

     private static ServerSocket server; //static ServerSocket variable
     private static int port = 9876; //socket server port on which it will listen

     public static void main(String args[]) throws IOException, ClassNotFoundException{
          server = new ServerSocket(port); //create the socket server object
          while(true){//keep listens indefinitely until receives 'exit' call or program terminates
               System.out.println("Waiting for the client request");
               Socket socket = server.accept(); //creating socket and waiting for client connection
               ObjectInputStream ois = new ObjectInputStream(socket.getInputStream()); //read from socket to ObjectInputStream object
               String message = (String) ois.readObject(); //convert ObjectInputStream object to String
               System.out.println("Message Received: " + message);
               ObjectOutputStream oos = new ObjectOutputStream(socket.getOutputStream()); //create ObjectOutputStream object
               oos.writeObject("Hi Client "+message); //write object to Socket
               ois.close(); //close resources
               oos.close();
               socket.close();
               if(message.equalsIgnoreCase("exit")) break; //terminate the server if client sends exit request
          }
          System.out.println("Shutting down Socket server!!");
          server.close(); //close the ServerSocket object
    }
}
```

Example 3

```
import java.io.IOException;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;
import java.net.InetAddress;
import java.net.Socket;
import java.net.UnknownHostException;

public class SocketClientExample {
     public static void main(String[] args) throws UnknownHostException, IOException, ClassNotFoundException, InterruptedException{
          InetAddress host = InetAddress.getLocalHost(); //get the localhost IP address, if case that server is running on some other IP
          Socket socket = null;
          ObjectOutputStream oos = null;
          ObjectInputStream ois = null;
          for(int i=0; i<5;i++){
               socket = new Socket(host.getHostName(), 9876); //establish socket connection to server
               oos = new ObjectOutputStream(socket.getOutputStream()); //write to socket using ObjectOutputStream
               System.out.println("Sending request to Socket Server");
               if(i==4)
                    oos.writeObject("exit");
               else
                    oos.writeObject(""+i);
               ois = new ObjectInputStream(socket.getInputStream()); //read the server response message
               String message = (String) ois.readObject();
               System.out.println("Message: " + message);
               ois.close(); //close resources
               oos.close();
               Thread.sleep(100);
        }
    }
}
```